

Design and Implementation of a BRAM-Banked Double-Buffered Matrix Multiplication Accelerator for Transformer Models on Edge FPGAs

Xinghang Tan
Zhejiang University
Hangzhou, China

tan_xinghang@zju.edu.cn

*Tee Hui Teo

Singapore University of Technology and Design
Singapore, Singapore

*Corresponding author: tthui@sutd.edu.sg

Abstract—Matrix multiplication is a critical computational bottleneck in modern Transformer-based AI systems, particularly within the self-attention and Feed-Forward Network (FFN) layers. Among different accelerator platforms, edge FPGAs provide notable power-efficiency benefits but are limited by their on-chip memory capacity and bandwidth. Most existing FPGA-based accelerators have focused on data center platforms or floating-point computation, whereas optimization of integer matrix multiplication for resource-constrained edge devices has received limited attention. This paper presents an optimized integer accelerator implemented via high-level synthesis (HLS), targeting Transformer-based inference models and specifically addressing the data movement bottleneck. The proposed design combines several key techniques—block RAM (BRAM) banking, double-buffering, and Advanced eXtensible Interface (AXI) bandwidth optimization—to enable parallel data access, overlap computation with data transfer, and improve bus utilization. Experimental results show that the proposed design improves throughput by 7.31× and reaches an effective computational performance of 22.83 GOPs, while reducing energy consumption to 14.6 % of the baseline. The results validate that an HLS-driven, system-level dataflow optimization can achieve both efficient and deployable acceleration on edge FPGAs, while further highlighting how processing system (PS) overhead constrains overall performance and informs future edge AI accelerator design.

Index Terms—AI Accelerator, FPGA, matrix multiplication, double-buffering, BRAM banking

I. INTRODUCTION

Matrix multiplication is a fundamental operation in both scientific computing and artificial intelligence systems. It appears not only in classical numerical algebra and signal processing but also underpins modern deep learning models—most notably the Transformer architecture, where the self-attention (Q/K/V) and FFN layers rely heavily on matrix multiplications [1]. As a result, achieving efficient matrix multiplication has long been a central research goal in both academia and industry.

Since the introduction of the systolic array architecture by Kung and Leiserson [2], dataflow-based parallel computation has become the classical paradigm for GEMM hardware acceleration. This concept not only laid the foundation for the development of application-specific integrated circuit (ASIC) and graphics processing unit (GPU) accelerators but also pro-

foundly influenced matrix multiplication implementations on FPGAs. Edge FPGA devices are more constrained in terms of compute capability, on-chip storage, and off-chip bandwidth. However, their advantages in reconfigurability, low power consumption, and real-time processing make them ideal choices for embedded inference and edge intelligence applications [3]. Therefore, enhancing the performance and energy efficiency of GEMM under restricted hardware conditions remains an important and challenging research problem.

In recent years, researchers have explored matrix multiplication optimization on FPGAs from multiple perspectives. Under resource-constrained conditions, performance bottlenecks often stem from limited on-chip storage and off-chip bandwidth. To address this, earlier research introduced analytical performance models for FPGA-based matrix multiplication [4] and the Roofline Model [5] for analyzing the performance ceilings imposed by compute- versus bandwidth-bound operations. Building on this foundation, de Fine Licht et al. [6] proposed the communication-avoiding GEMM framework, emphasizing tiling and double-buffering to hide communication latency and overlap computation with data transfer. Taka et al. [7] further presented a systematic GEMM design methodology with formalized schemes for BRAM banking and double-buffering. More recently, Mhatre et al. [8] analyzed how double-buffering affects BRAM utilization on the Versal platform and identified the conditions under which calculation shifts from compute-bound to memory-bound behavior. These studies provide valuable design insights and highlight the importance of balancing resources and bandwidth on FPGAs.

While extensive research has been conducted on resource-bandwidth trade-offs on FPGAs, most studies focus on data center devices such as Versal and Stratix. There remains limited investigation into how to systematically balance bandwidth utilization, computational density, and storage overhead on resource-constrained edge FPGAs. Existing work [3], which utilized MAESTRO [9] and related analysis frameworks, implemented a tile-based matrix multiplication accelerator combining two-level tiling, on-chip buffering, and a systolic-like compute array, achieving notable performance gains in DistilBERT inference. This design provides a direct baseline

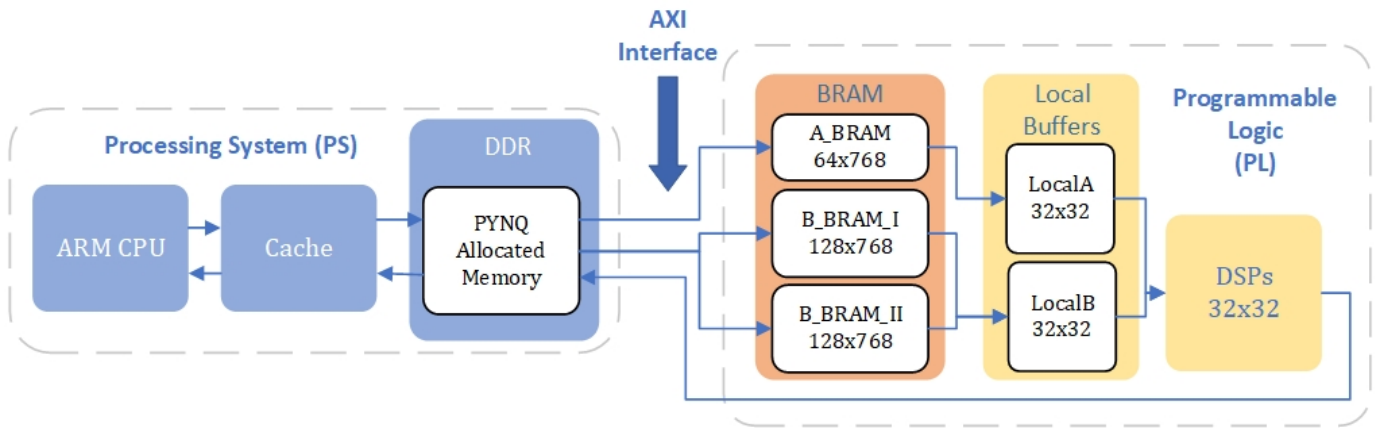


Fig. 1. Overall architecture of the proposed FPGA accelerator.

for matrix multiplication optimization on edge FPGAs.

This paper presents a matrix multiplication accelerator efficiently implemented using HLS that integrates BRAM banking, double-buffering, and AXI bandwidth optimization to enhance data reuse and memory-level parallelism. Implemented on the AMD Kria KV260 platform, the accelerator achieves $7.31\times$ higher throughput compared with the baseline while consuming only 14.6 % of its energy. These results demonstrate the effectiveness of system-level dataflow optimization on edge FPGAs and validate the practicality of the HLS-based development flow proposed in [3]. The design provides reusable insights for future low-power FPGA-based AI accelerators.

II. METHODOLOGY

In the Transformer architecture, the matrix multiplications within the attention mechanism and the FFN typically involve multiplying a relatively small matrix A with a larger matrix B . Profiling the baseline design in HLS reveals that only 2 % of the total clock cycles are devoted to actual matrix multiplication, while 32 % are spent loading matrices A and B from DDR to BRAM and 59 % on transferring data from BRAM to the local buffer during each tile computation. This imbalance highlights a severe data-transfer bottleneck. To address this problem, we propose an optimization strategy that combines three techniques: parallel data movement, double-buffering, and AXI bus bandwidth enhancement.

Fig. 1 illustrates the dataflow in the proposed design. On the programmable logic (PL) side, the tile size is constrained by the available Digital Signal Processor (DSP) resources; to balance design complexity and performance, it is set to 32×32 .

A. Parallelizing Data Movement

In general designs, the BRAM storage space for matrices A and B is not partitioned. However, under the AMD architecture, each physical BRAM block (18 Kb) on the FPGA possesses independent read/write ports. If the array size to be stored exceeds a single physical BRAM capacity, multiple physical BRAMs are typically concatenated into a single larger BRAM,

retaining only one read/write port. Consequently, data transfer speed—both from DDR to BRAM and from BRAM to the local buffer—is severely limited in common designs. Although the local buffer uses a complete partitioning mode, allowing simultaneous access to 32×32 registers, its performance remains constrained by the BRAM access bottleneck.

To solve this problem, we fully leverage the inherent capability of the BRAMs by using the `array_partition` HLS optimization directive to cyclically partition the BRAMs storing matrices A and B . For example, we use `#pragma HLS ARRAY_PARTITION variable = A_bram type = cyclic factor = 32 dim = 2` to realize the partition of $BRAM_A$. Specifically, matrix data is partitioned column-wise, where columns with the same column index modulo 32 are stored in the same BRAM block. This results in 32 BRAM blocks being used for matrix A and 64 BRAM blocks for matrix B (due to the double-buffering technique). The cyclic mode enables 32-way parallel data reading when transferring any tile. Furthermore, the column-wise partitioning allows the BRAMs to store data in parallel when receiving 128-bit packed data transferred from the DDR.

Fig. 2 illustrates this BRAM banking scheme, where data from different matrix columns are stored in the physical BRAMs indicated by the corresponding colors. Columns 0,32,64,... are stored in Physical BRAM 0; columns 1,33,65,... are stored in Physical BRAM 1; and so on. In this way, the entire matrix data is cyclically stored across 32 BRAMs, ensuring that the bandwidth of every Physical BRAM is effectively utilized.

B. Double-Buffering

Double-buffering is a technique that uses two buffers to operate alternately, eliminating access competition by separating the buffers accessed by the producer (data loading) and the consumer (computation), thus ensuring smooth and efficient data exchange. Given the large size of matrix B in the Transformer architecture, it is typically loaded into the BRAM in column blocks, with the next block loaded only after the computation of the current block. To overlap the loading and computation stages, we use the HLS `DATAFLOW` pragma to

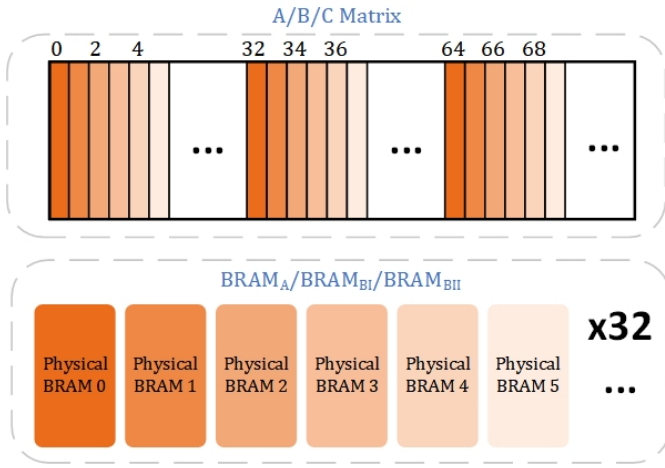


Fig. 2. Separates matrix data to different BRAMs.

achieve this functionality. For the loading of matrix B , we apply double-buffering by dividing the original $BRAM_B$ into two buffers, $BRAM_{BI}$ and $BRAM_{BII}$, each of which stores half of the block columns of matrix B .

The process operates as follows: When $BRAM_{BI}$ is fully loaded, the compute unit begins consuming its data, while $BRAM_{BII}$ simultaneously begins loading the next block. When the compute unit has finished all data in $BRAM_{BI}$ and $BRAM_{BII}$ is fully loaded, the compute unit switches to consume data from $BRAM_{BII}$. This mechanism allows $BRAM_{BI}$ and $BRAM_{BII}$ to alternately swap roles as the producer and consumer, greatly increasing the parallelism between data loading and computation without increasing the total BRAM resource usage (beyond the initial splitting).

C. Increasing AXI4-Full Bus Bandwidth

AXI4 is a high-performance, high-frequency interconnection protocol widely adopted in modern System-on-Chip (SoC) designs. It provides a unified communication interface for integrating different functional IP cores, simplifying the integration and verification processes of complex chips. The AXI4-Full bus supports a maximum width of 128 bits. However, when data types like int8 or int32 are used for AXI bus transfers in Vitis HLS, the bus width is automatically configured to match the data type width and cannot be widened during synthesis. To fully utilize the AXI bus bandwidth resources, we implement data packing and unpacking for inputs and outputs:

- When loading matrices A and B from DDR into the BRAM, we use the unsigned int 128 data type to transfer 16 int8 elements per clock cycle. The data is written into the BRAM leveraging the grouped BRAM characteristics mentioned in Section 2.1 to achieve 16-way parallel writing.
- When writing the outputs of a single tile from the local buffer back to DDR, we use the unsigned int 128 data type to transfer 4 int32 elements per clock cycle.

With the expanded bandwidth, the BRAM banking technique described in Section II-A can be leveraged to enable

concurrent loading of matrices A and B , allowing one AXI beat—corresponding to 16 matrix elements—to be written within a single clock cycle.

III. PERFORMANCE EVALUATION

For evaluation purposes, this study adopted the same benchmarking settings as [3], including the measurement methodology, matrix dimensions, dataflow pattern, and hardware platform (AMD Kria KV260). All experiments were performed under identical hardware configurations, enabling a direct comparison with the baseline design.

Table I summarizes the co-simulation results obtained from Vitis HLS. The optimized design shows a substantial performance gain over the baseline for a matrix multiplication of size $(64,768) \times (768,768)$, corresponding to a DistilBERT attention case. The observed speedup primarily stems from reduced transfer time when copying matrices A and B from DDR to BRAM (CopyA/CopyB) and when loading data from BRAM to the local buffer (LoadA/LoadB), which directly reflects the intended effects of our optimization strategy. Benefiting from the widened AXI bus and BRAM banking, the theoretical speedups of CopyA/CopyB, LoadA/LoadB, and WriteC reach 16 $\times$, 32 $\times$, and 4 $\times$, respectively. The simulation results are consistent with the theoretical estimation.

TABLE I
CLOCK CYCLES RESULT @ 100MHz

	Baseline Design	Proposed Design	Speedup
Total	2,015,050	130,030	15.50 $\times$
CopyA	49,157	3076	15.98 $\times$
CopyB	601,377	-	-
LoadA	1,180,800	39,168	30.15 $\times$
LoadB	1,180,800	39,168	30.15 $\times$
Compute	40,320	40,320	1 $\times$
WriteC	139,824	35,328	3.96 $\times$

Note—CopyA/CopyB: DDR-to-BRAM transfer; LoadA/LoadB: BRAM-to-local-buffer transfer; Compute: matrix multiplication; WriteC: write back to DDR.

The resource utilization after the Vivado implementation is presented in Table II. Compared to the baseline design, the proposed design shows a reduction in BRAM utilization, along with a minor increase in Flip-Flops (FF) and Look-Up Tables (LUT) usage.

TABLE II
RESOURCE UTILIZATION FOR TILED MATMUL ACCELERATOR ON XCK26 (KV260) @ 100 MHz

Resource	Used	Available	Utilization
BRAM	102	144	70.83%
DSP48E1	1038	1248	83.17%
FF	104067	234240	44.43%
LUT	98054	117120	83.72%

The experimental results obtained from deploying the optimized design on the AMD Kria KV260 platform are summarized in Table III. For a FFN case of $(64,768) \times (768,3072)$, the proposed FPGA design achieves a latency of 13.2 ms, providing a 7.31 $\times$ speedup over the baseline (96.7 ms) and an

effective throughput of approximately 22.83 GOPs. The energy evaluation further shows that the optimized design consumes only 0.0488 J to complete the same matrix multiplication, representing 14.6 % of the baseline's energy consumption.

In addition to the baseline design, we compare the proposed accelerator with a CPU implementation on the ARM Cortex-A53 optimized using PyTorch [3], as well as another edge-FPGA implementation on the Xilinx Zynq UltraScale+ MPSoC ZU3EG A484 that performs 32-bit integer matrix multiplication on a 256×256 square matrix [10], to enable a more comprehensive throughput evaluation.

TABLE III
PERFORMANCE COMPARISON AMONG DIFFERENT IMPLEMENTATIONS

Design	Throughput (GOPs)	MOPs/DSP	Energy (J)
Baseline	3.12	2.50	0.335
Proposed	22.83	18.29	0.049
ARM Cortex-A53 [3]	0.45	-	2.235
ZU3EG (INT32) [10]	0.90	2.50	-

We implemented our design within a DistilBERT model using the Jupyter Lab environment provided by PYNQ, replacing the original Q/K/V matrix multiplication layers. In this scenario, each inference involves 18 matrix multiplications of size $(64,768) \times (768,768)$. Table IV shows that the proposed design achieves a 6.90 $\times$ speedup over the baseline, with each computation taking only 3.93 ms. These results further demonstrate that our method can substantially improve the inference speed of Transformer models on edge devices while maintaining computational accuracy.

TABLE IV
PERFORMANCE ON A DISTILBERT INFERENCE MODEL @ 100MHz

	Baseline Design	Proposed Design
Compute time (ms)	430.76	70.66
Average time per call (ms)	23.93	3.93

Despite the promising speedup, the experiments also reveal certain system-level limitations. When the throughput measurement includes data processing on the PS side, the throughput decreases from 22.83 GOPs to 13.52 GOPs. This degradation was relatively small in [3] (from 3.12 GOPs to 2.85 GOPs) but becomes more evident in our optimized design. This finding underscores a key challenge in matrix multiplication acceleration on edge FPGAs: while offloading computation to the PL substantially accelerates execution, additional data-handling overhead on the PS side can limit the overall throughput. The same trend is observed in DistilBERT inference, where quantization and dequantization account for 76 % of the total runtime, and PL side computation contributes only 24 %, primarily for accelerating the Q/K/V matrix multiplications. These observations highlight that effective coordination between the PS and PL is essential to fully realize the performance potential of edge FPGA-based Transformer acceleration.

IV. CONCLUSION

This paper presents an HLS-based optimization framework for matrix multiplication on resource-constrained edge FPGAs.

Profiling on the AMD Xilinx Kria KV260 platform revealed that data movement is the primary performance bottleneck. To address this issue, a coordinated strategy was implemented, combining BRAM banking, double-buffering, and AXI4-Full bandwidth optimization. Experimental results show that the optimized integer accelerator achieves 7.31 $\times$ lower latency and a peak throughput of 22.83 GOPs compared with the baseline, while consuming only 14.6 % of its energy. These results validate that system-level dataflow optimization, enabled by efficient HLS design iteration, can effectively improve computational efficiency on FPGAs under strict resource constraints. Future work will explore hardware-software co-design to enhance the end-to-end acceleration pipeline, with a particular focus on optimizing host-side data preprocessing to further improve Transformer inference performance on edge devices.

ACKNOWLEDGMENT

The authors thank the support provided by the SUTD-ZJU IDEA Visiting Professor Grant (SUTD-ZJU (VP) 202103) and the SUTD-ZJU Thematic Research Grant (SUTD-ZJU (TR) 202204). This project was undertaken as part of the DEEPS-AI (Design, Engineering, Education, Psychology, Science - Artificial Intelligence) initiative, aimed at multidisciplinary experiential learning.

REFERENCES

- [1] S. Lu, M. Wang, S. Liang, J. Lin, and Z. Wang, "Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer," in *2020 IEEE 33rd International System-on-Chip Conference (SOCC)*. IEEE, 2020, pp. 84–89.
- [2] H. T. Kung and C. E. Leiserson, "Systolic arrays (for vlsi)," in *Sparse Matrix Proceedings 1978*, vol. 1. Society for industrial and applied mathematics Philadelphia, PA, USA, 1979, pp. 256–282.
- [3] R. Li and S. Chen, "Design and implementation of an fpga-based tiled matrix multiplication accelerator for transformer self-attention on the xilinx kv260 som," *arXiv e-prints*, pp. arXiv–2503, 2025.
- [4] C. Y. Lin, H. K.-H. So, and P. H. Leong, "A model for matrix multiplication performance on fpgas," in *2011 21st International Conference on Field Programmable Logic and Applications*. IEEE, 2011, pp. 305–310.
- [5] S. W. Williams, A. Waterman, and D. A. Patterson, "Roofline: An insightful visual performance model for floating-point programs and multicore architectures," Technical Report UCB/EECS-2008-134, EECS Department, University of ..., Tech. Rep., 2008.
- [6] J. de Fine Licht, G. Kwasniewski, and T. Hoefler, "Flexible communication avoiding matrix multiplication on fpga with high-level synthesis," in *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2020, pp. 244–254.
- [7] E. Taka, D. Gourounas, A. Gerstlauer, D. Marculescu, and A. Arora, "Efficient approaches for gemm acceleration on leading ai-optimized fpgas," in *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2024, pp. 54–65.
- [8] K. M. Mhatre, V. G. P. Mulleti, C. J. Bansil, E. Taka, and A. Arora, "Performance analysis of gemm workloads on the amd versal platform," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 46–46.
- [9] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 754–768.
- [10] J. A. Belloch, G. León, J. M. Badía, A. Lindoso, and E. San Millán, "Evaluating the computational performance of the xilinx ultrascale+ eg heterogeneous mpso," *The Journal of Supercomputing*, vol. 77, no. 2, pp. 2124–2137, 2021.